COURS 3: SOCKETS

Plan

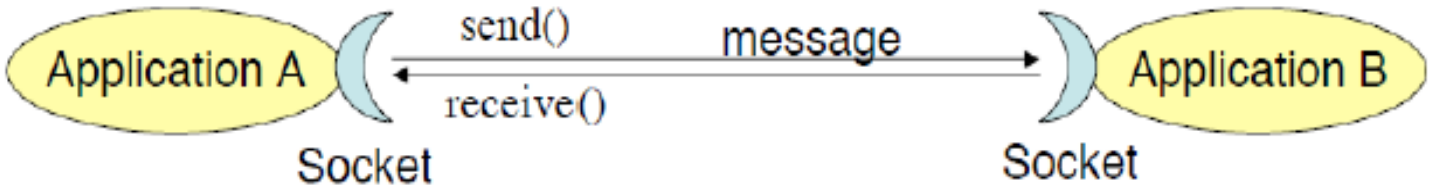
I – Programmation réseau

- Introduction et rappels sur les sockets
- Sockets TCP et UDP
- Pool de connexion

I- Introduction et rappels sur les sockets

Socket : connecteur réseau

- Communication inter processus à distance
- Envoi et réception de messages via un réseau
- Socket = interface de programmation (API) avec les services du système d'exploitation pour exploiter les services de communication du système (local ou réseau)
- Une socket est un demi-point de connexion d'une application



I- Introduction et rappels sur les sockets

Couche	Nom	Rôle	Protocoles / exemples
Couche 7	Application	Interface avec les logiciels et services utilisateur	NFS
Couche 6	Présentation	Représentation indépendante des données (format, codage, chiffrement, compression)	XDR
Couche 5	Session	Établissement et maintien des sessions	RPC
Couche 4	Transport	Liaison de bout en bout entre applications, fragmentation, éventuellement vérification (fiabilité)	**TCP, UDP, Multicast**
Couche 3	Réseau	Adressage et routage entre machines	IP
Couche 2	Liaison	Encodage pour l'envoi, détection d'erreurs, synchronisation	Ethernet (couche physique), fibre, Wi-Fi
Couche 1	Physique	Support de transmission lui-même (câbles, signaux électriques, ondes)	

I- Introduction et rappels sur les sockets

Liaison par flux : Socket/ServerSocket (TCP)

- Connecté : protocole de prise de connexion (**lent**)
- =>communication uniquement point à point
- **Sans perte** : un message arrive au moins un fois
- **Sans duplication** : un message arrive au plus une fois
- Avec fragmentation : les messages sont coupés
- **Ordre respecté**
- Communication de type téléphone

Liaison par datagram : DatagramSocket/DatagramPacket (UDP)

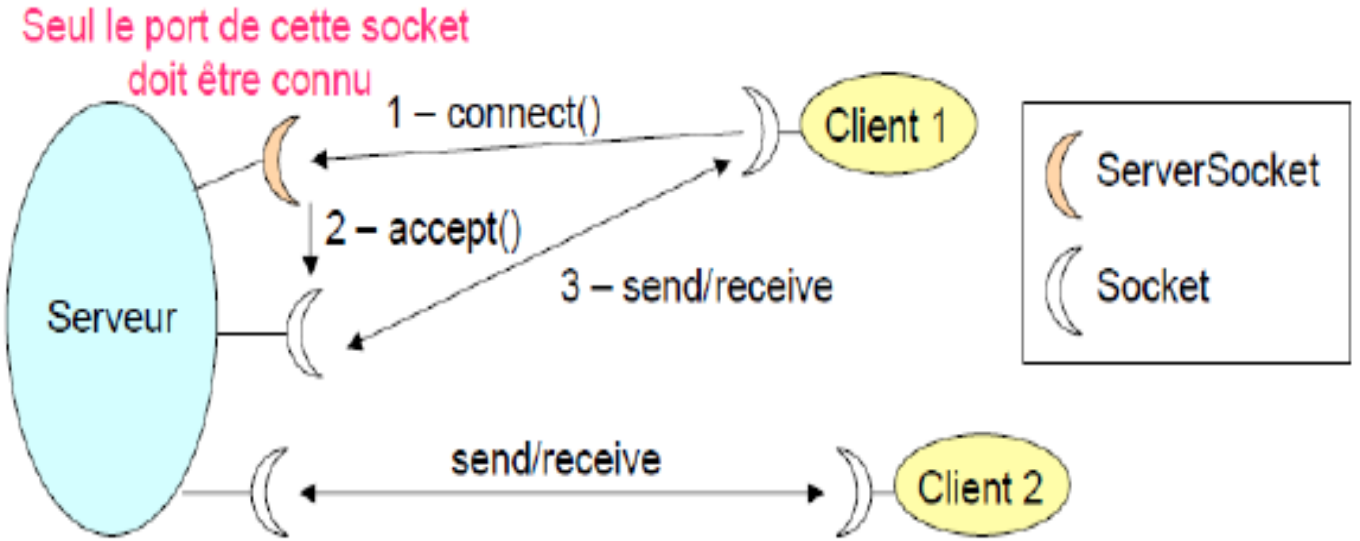
- Non connecté : pas de protocole de connexion (**plus rapide**)
- **Avec perte** : l'émetteur n'est pas assuré de la délivrance
- **Avec duplication** : un message peut arriver plus d'une fois
- Sans fragmentation : les messages envoyés ne sont jamais coupés
- =>soit un message arrive entièrement, soit il n'arrive pas
- **Ordre non respecté**
- Communication de type courrier

	Fiabilité	Ordre	Connexion
	Est-ce que les messages sont garantis sans erreur ?	Est-ce que les messages arrivent dans le même ordre que celui de leur émission ?	Les échanges de données sont-ils organisés en cx ?
TCP	Oui	Oui	Oui
UDP	Non	Non	Non

Une socket est identifiée par :

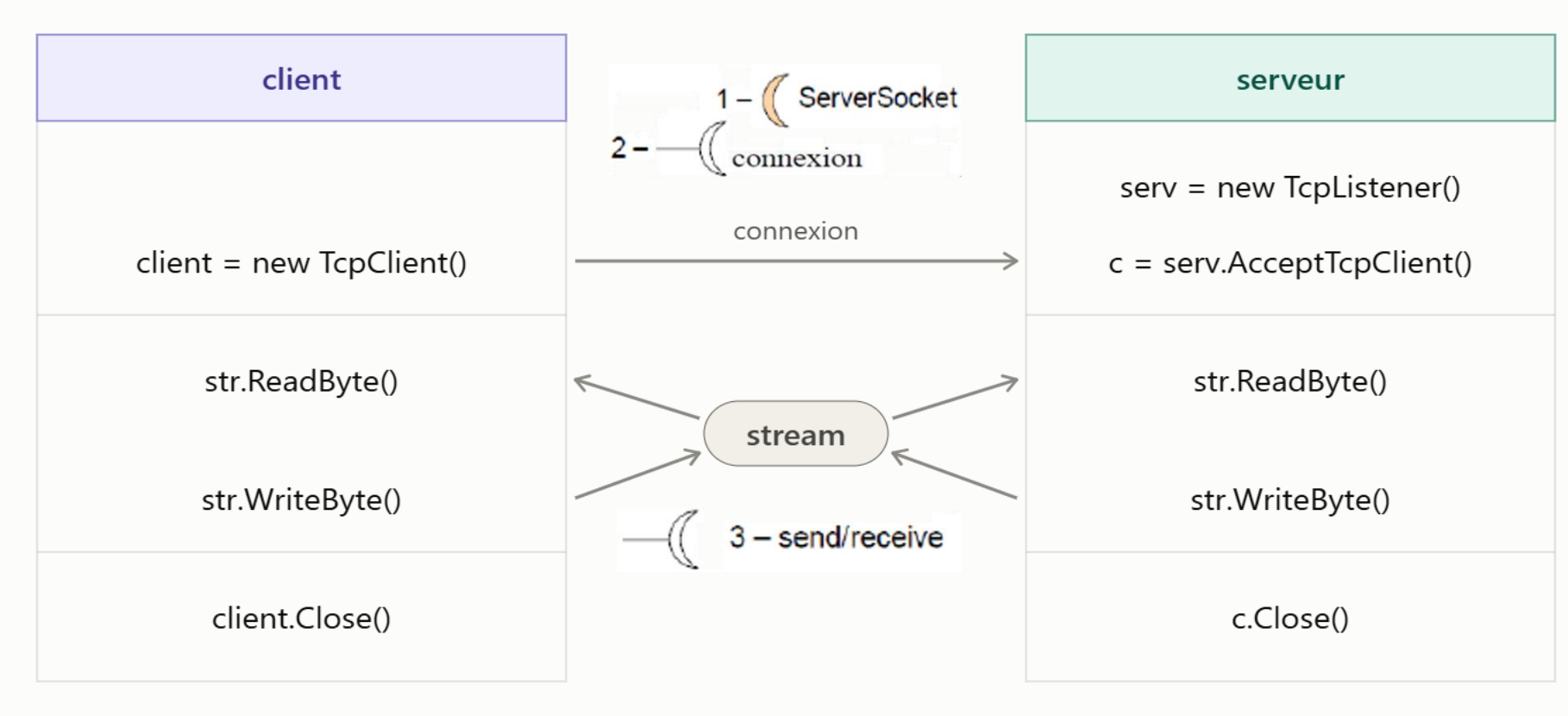
- **Adresse IP** : une des adresses de la machine (ou toutes)
- **Port** : attribué automatiquement ou choisi par le programme
- **Adresse de Socket** = Adresse IP + port
- Une socket communique avec une autre via à son adresse
- **Flux** : une socket se connecte à une autre socket via son adresse de socket
- **Datagram** : une socket envoie/reçoit des données à/d'une autre socket identifiée par son adresse de socket

TCP – Fonctionnement

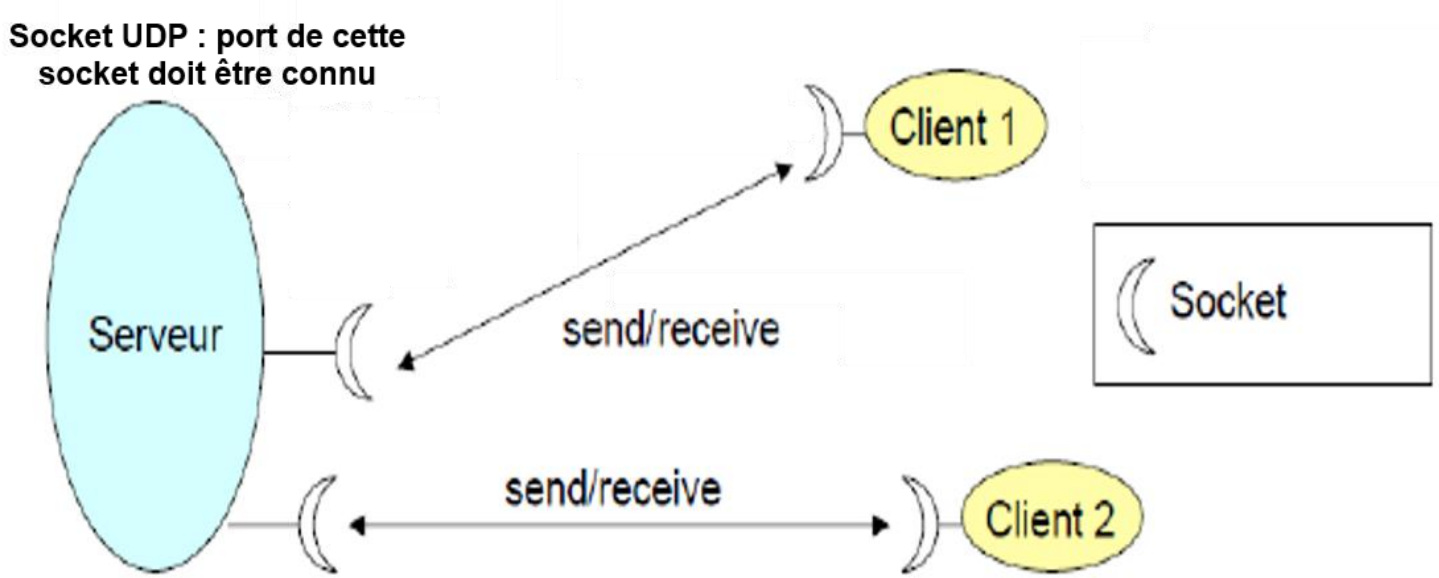


- Serveur crée une ***socket*** et attend une demande de connexion
- Client envoie une demande de connexion
- Serveur accepte connexion
- Dialogue client/serveur en mode flux
- Fermeture de connexion à l'initiative du client ou du serveur

TCP – Fonctionnement

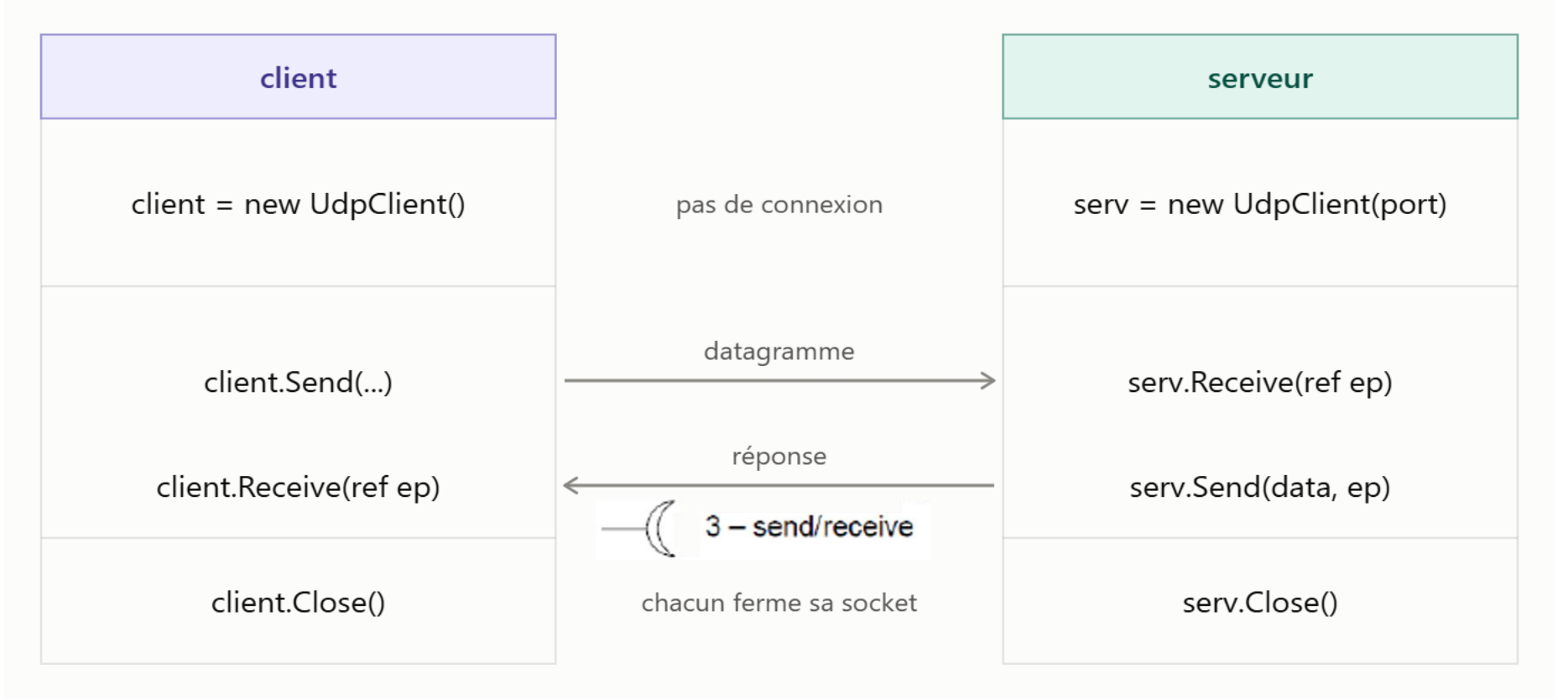


UDP – Fonctionnement



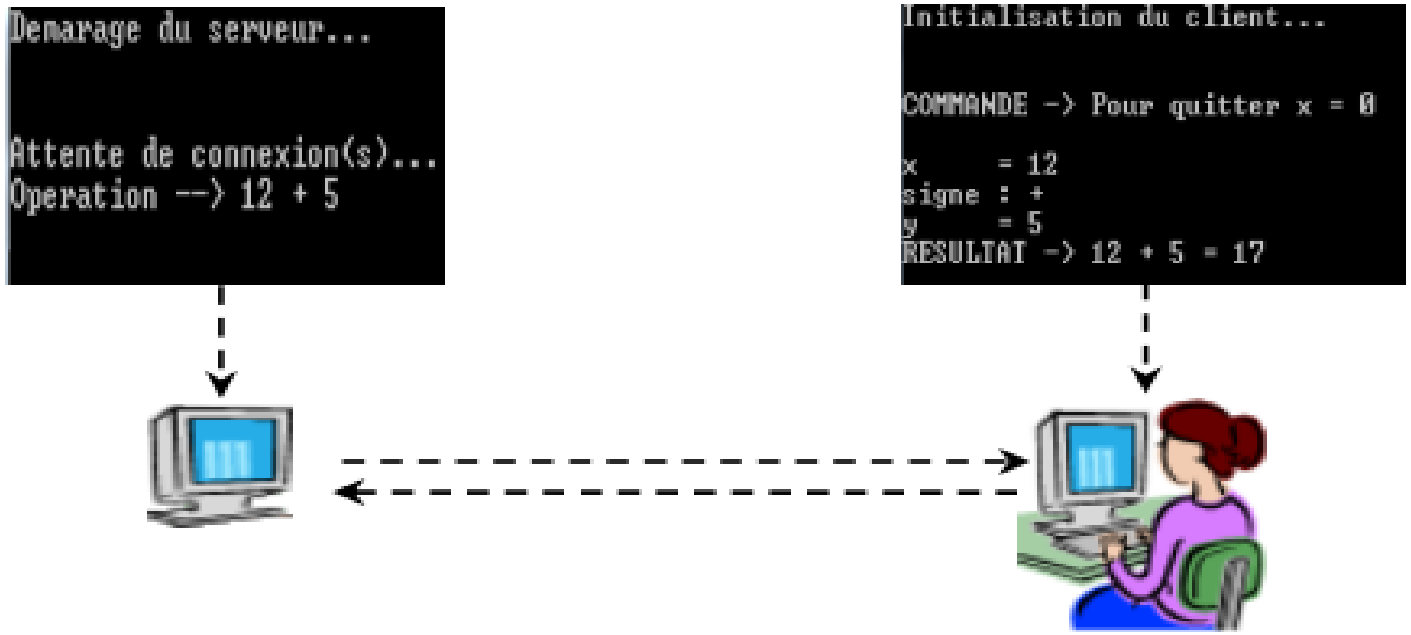
- Le serveur crée une ***socket***, la lie à un port et attend un datagramme
- Le client crée une ***socket*** et envoie un datagramme au serveur ((@ IP + port)
- Le serveur reçoit le datagramme (avec l'adresse et le port de l'expéditeur)
- Dialogue client/serveur en mode datagramme (chaque message est indépendant, sans garantie de livraison)
- Pas de fermeture de connexion : chacun ferme simplement sa ***socket*** localement

UDP – Fonctionnement



Sockets UDP - Exemple

- Soit à réaliser une calculatrice Windows Standard avec une architecture Client / Serveur.
- La communication entre le Client et le Serveur se fait par socket UDP.



UdpServeur : Application Console

Client : Application Console

Projet UdpServeur – Classe serveur

```
using System;
using System.Net;
using System.Net.Sockets;

namespace UdpServeur
{
    public class serveur {
        public static void Main(string[] args) {
            private int P = 8080;//port de connexion
            private IPEndPoint ip = null;

            //Demarage su serveur
            Console.WriteLine("Demarage du serveur...");
            UdpClient serveur = new UdpClient(P);
            private bool loop = true;

            while (loop) {
                //Attente de connexion
                Console.WriteLine("Attente de connexion(s)...");
                byte[] tmp = serveur.Receive(ref ip);
                //Conversion des bytes en string (chaine de caractere)
                string data = new System.Text.ASCIIEncoding().GetString(tmp);
                //Dechifrage de la commande --> ip:port:command
                string[] cmd = data.Split(new char[] { ':' });

                //adresse ip
                string host = cmd[0];

                //port de connexion
                int port = Int32.Parse(cmd[1]);
```

Projet UdpServeur – Classe serveur (Suite)

```
        //recuperation de nb1 du signe et de nb2
        private int nb1 = Int32.Parse(cmd[2]);
        private string signe = cmd[3];
        private int nb2 = Int32.Parse(cmd[4]);

        private int resultat = 0;
        Console.WriteLine("Operation --> " + nb1 + " " + signe + " " + nb2);

        //execution de la commande
        switch (signe)
        {
           case "+": resultat = nb1 + nb2; break;
           case "-": resultat = nb1 - nb2; break;
           case "*": resultat = nb1 * nb2; break;
           case "/": resultat = nb1 / nb2; break;
           case "<>": resultat = 0; loop = false; break;
           default: resultat = 0; break;
        }//fin du switch
        private string reponse = nb1 + " " + signe + " " + nb2 + " = " + resultat;

        //conversion du resultat en bytes
        byte[] rep = System.Text.Encoding.ASCII.GetBytes(reponse.ToCharArray());
        serveur.Send(rep, rep.Length, host, port);

        } //fin du while

        //arrêt du serveur
        Console.WriteLine("Arret du serveur...");
        Console.WriteLine("Arret du serveur... *Entree* pour terminer.");
        Console.ReadLine();
    } } }
```

Projet Client – Classe UDPClient

```
using System;
using System.Net;
using System.Net.Sockets;

namespace Client {
    public class UDPClient {
        public static void Main(string[] args)
        {
            Console.WriteLine("Initialisation du client...");
            client proc = new client(8081, "127.0.0.1", 8080);
            string x, y, signe, command, resultat = "";
            while (true)
            {
                Console.WriteLine(" ");
                Console.WriteLine("COMMANDE -> Pour quitter x = 0\n");
                //entrer x
                Console.Write("x     = "); x = Console.ReadLine();
                //terminer si x = 0
                if (Int32.Parse(x) == 0) break;
                //entrer le signe
                Console.Write("signe : "); signe = Console.ReadLine();
                //entrer y
                Console.Write("y     = "); y = Console.ReadLine();
                command = x + ":" + signe + ":" + y;
                proc.Execute(command, ref resultat);
                Console.WriteLine("RESULTAT -> " + resultat);
            }
        }
    }
}
```

Port du client

Adresse IP du serveur

Port du serveur

Projet Client – Classe client

```
using System;
using System.Net;
using System.Net.Sockets;
namespace Client {
    public class client {
        private int clientPort = -1;        //port du client
        private int serveurPort = -1;       //port du serveur
        private string serveurHost = null; //adresse du serveur
        private UdpClient Client = null;    //client
        //constructeur
        public client(int clientPort, string serveurHost, int serveurPort) {
            this.clientPort = clientPort;
            this.serveurHost = serveurHost;
            this.serveurPort = serveurPort;
            this.Client = new UdpClient(clientPort); }
        public void Close() { this.Client.Close(); }
        public bool Execute(string command, ref string resultat)
        {   bool ok = true;
            //commande a envoyer
            string req ="127.0.0.1" + ":" + this.clientPort + ":" + command;
            //conversion de la commande en byte
            byte[] rq = System.Text.Encoding.ASCII.GetBytes(req.ToCharArray());
            //envoi de la commande
            Client.Send(rq, rq.Length, this.serveurHost, this.serveurPort);
            //variable de reception du resultat
            IPEndPoint ip = null;
            //reception
            byte[] r = Client.Receive(ref ip);
            //conversion
            resultat = System.Text.Encoding.ASCII.GetString(r);
            ok = !resultat.Equals(0);
            return ok;
} } }
```

Adresse IP du client

TCP et UDP

Asynchronisme :

- TCP : `Read` et `Write` synchrones sur le flux, ou asynchronisme avec `BeginRead/EndRead` et `BeginWrite/EndWrite`
- UDP : `Send` et `Receive` synchrones, ou asynchronisme avec `BeginSend/EndSend` et `BeginReceive/EndReceive`

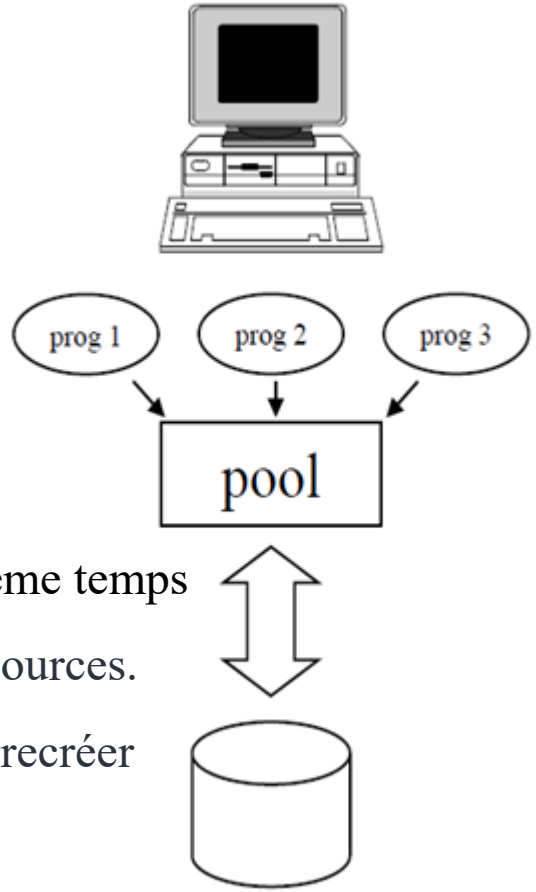
Autres protocoles de transport :

- Classe Socket : en C#, la classe Socket appartient à l'espace de noms `System.Net.Sockets`.
- Elle permet de créer et de gérer des communications réseau avec :
  1. TCP : communication fiable, ordonnée, basée sur une connexion.
  2. UDP : communication par datagrammes, sans garantie de livraison.
  3. Multicast IP : envoi de données à un groupe de récepteurs qui ont rejoint un groupe multicast avec UDP.
- La classe Socket offre un contrôle plus direct sur les opérations réseau, contrairement à `TcpClient`, `TcpListener` ou `UdpClient`.

Pool de connexion

Connexions réseau coûteuse

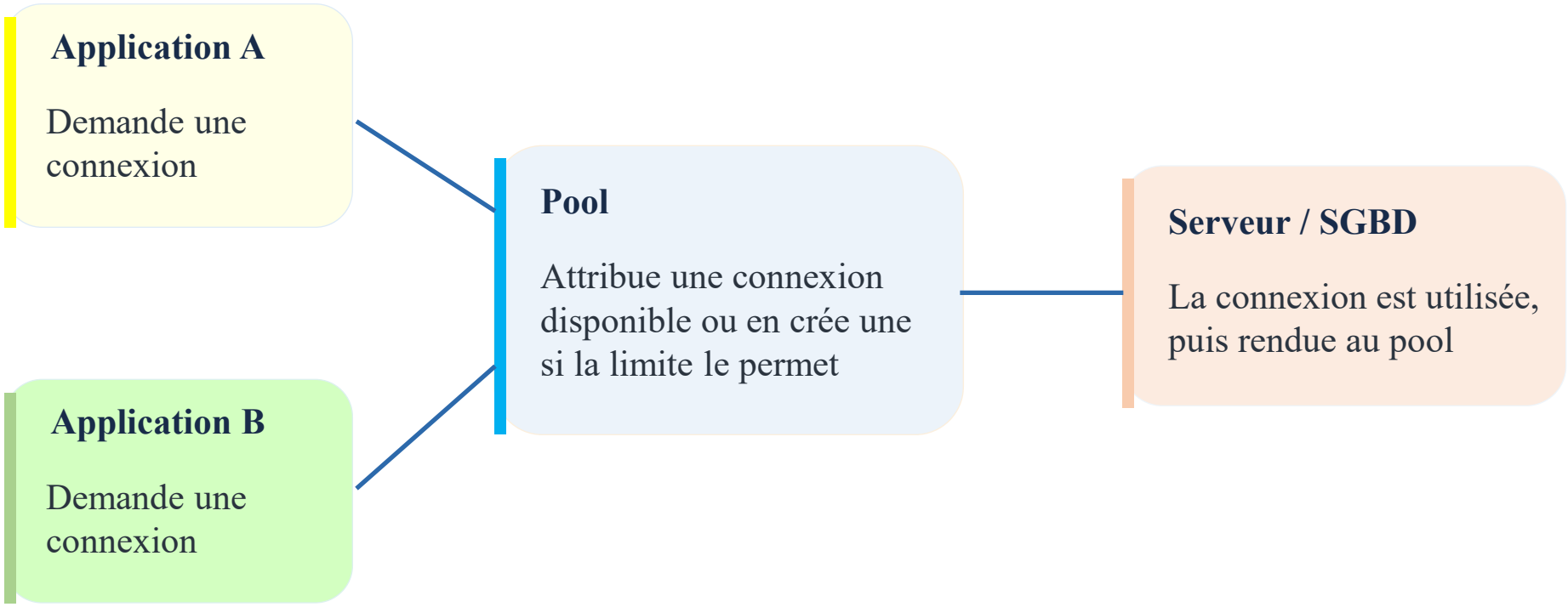
- En ressources occupées
- En temps mis pour ouvrir/fermer les connexions



But du *pool* : mutualiser les connexions entre deux machines.

- Tous les programmes n'ont pas besoin de toutes les connexions en même temps
- Ouvrir et fermer une connexion peut consommer du temps et des ressources.
- Un pool conserve un ensemble de connexions réutilisables au lieu de recréer une connexion.
- Le pool limite ou régule le nombre de connexions simultanées et améliorer les performances.
- Exemple : le pool de connexions à un SGBD, géré par le fournisseur de données.

Principe de fonctionnement d'un pool de connexion



Pool :

1. Améliore la réutilisation,
2. Il faut gérer :
   - les limites du pool,
   - les délais d'attente,
   - la libération correcte des ressources.